

8-1. Calibration 평가 방법론

신경망 분류기의 **보정(Calibration)**은 왜 중요할까요? 최신 딥러닝 모델은 정확도는 높지만 예측 확률이 실제 확률과 일치하지 않는 과신 경향을 보이는데, 이는 의사결정 시스템에 오류를 야기할 수 있습니다. 따라서 모델의 예측 확률이 실제 확률 분포를 얼마나 잘 반영하는지 측정하고 보정하는 과정이 필수적입니다.

1. 신경망 분류기의 보정(Calibration) 이해하기: 정확도와 신뢰도 사이의 균형

신경망 분류기의 **보정(Calibration)**은 모델의 예측 확률이 실제 확률을 얼마나 잘 반영하는지를 측정하고 개선하는 과정이다. 최신 딥러닝 모델은 높은 정확도를 자랑하지만, 종종 예측 확률에 있어 과신하는 경향을 보인다. 이는 실제 확률 분포와 모델의 예측 확률 간의 불일치를 야기하며, 특히 의사결정 시스템에서 오류를 발생시킬 수 있다. 따라서 모델의 예측이 얼마나 신뢰할 수 있는지 이해하고, 필요하다면 보정하는 과정이 필수적이다.

1.1. 보정(Calibration)의 중요성 및 배경

- **보정의 정의:** 모델이 예측한 확률이 실제 사건의 발생 확률과 일치하도록 만드는 과정이다.
 - 예를 들어, 모델이 80%의 확률로 비가 올 것이라고 예측했다면, 실제로 그러한 예측이 100번 이루어졌을 때 약 80번은 비가 와야 한다.
- **보정의 역사:** 보정 개념은 1960년대 기상학자들이 날씨 예보의 정확도를 높이기 위해 처음 고안했다.
- **머신러닝에서의 보정:** 최근 머신러닝 분야에서 보정의 중요성이 부각되고 있다. 이는 모델의 확률적 예측을 실제 예측처럼 활용하여 시뮬레이션이나 의사결정에 더 유용하게 만들기 위함이다.
- **nPlan에서의 보정:** nPlan은 건설 프로젝트의 완료 시점을 예측하는 데 머신러닝 모델을 사용하며, 결정론적 예측 대신 확률적 예측을 통해 잠재적 결과의 분포를 파악하는 데 보정이 필수적이다.
 - 모델의 확률적 출력이 실제 확률 분포를 잘 반영해야 한다.
 - 이를 통해 모델을 예측 엔진으로 활용하고, 더 나은 의사결정을 내릴 수 있다.
- **현대 딥러닝 모델의 과신 경향:** 최신 신경망 모델은 정확도 향상에 집중하면서도 예측 확률의 신뢰도를 놓치는 경우가 많다.

- 이러한 과신은 실제 데이터셋에서는 벤치마크 데이터셋에서 보이는 성능만큼 나오지 않는 이유 중 하나이다.
- 특히 복잡한 모델 구조와 학습 기법(예: 배치 정규화)이 과신에 기여하는 것으로 알려져 있다.

- **보정이 필요한 이유:**

- **의사결정 시스템:** 모델의 예측 확률이 실제 확률을 반영해야 의사결정 시스템에서 신뢰할 수 있는 정보를 제공할 수 있다.
- **불확실성 추정:** 모델의 자신감 수준과 내재된 불확실성(epistemic uncertainty, aleatoric uncertainty)을 이해하는 데 도움을 준다.
- **고영향 시나리오:** 의료 진단과 같이 결과에 큰 영향을 미치는 분야에서는 작은 확률 오차도 중요하게 작용한다.

1.2. 보정의 개념과 측정 방법

1.2.1. 보정의 기본 개념

- **확률적 예측의 의미:** 모델의 확률적 출력은 특정 결과가 발생할 가능성을 나타내야 한다.
 - 이진 분류의 경우, 20% 확률로 예측된 클래스는 실제 데이터에서 20%의 빈도로 나타나야 한다.
- **현대 모델의 문제점:** 현대 신경망은 소프트맥스(softmax) 출력이 실제 확률을 제대로 반영하지 못하고 과도하게 확신하는 경향을 보인다.
 - 낮은 확률의 사건을 실제보다 훨씬 높은 확신으로 예측하는 경향이 있다.
- **정확도 지표의 한계:** F1 점수, 정밀도, 재현율과 같은 일반적인 정확도 지표는 모델의 과신을 오히려 장려할 수 있다.

1.2.2. 보정 측정 방법: 신뢰도 다이어그램 (Reliability Diagram)

- **신뢰도 다이어그램이란:** 모델이 예측한 확률과 실제 발생 확률 간의 관계를 시각화하는 그래프이다.
- **작동 방식:**
 1. 모델의 예측 확률을 여러 구간(bin)으로 나눈다.
 2. 각 구간별로 실제 정답(positive)의 비율을 계산한다.
 3. 이 비율을 바탕으로 그래프를 그린다.

- **완벽한 보정 모델:** 완벽하게 보정된 모델은 예측 확률과 실제 확률이 일치하므로 대각선 ($y=x$) 형태의 직선을 보인다.
- **실제 모델의 그래프:** 대부분의 모델은 대각선에서 벗어나며, 과신 또는 과소신 경향을 보인다.
 - **과소신(Underconfident):** 예측 확률보다 실제 정답 비율이 높은 경우 (그래프가 대각선 위로 올라감). 예를 들어, 20% 확률로 예측했지만 실제로는 40%가 정답이었다.
 - **과신(Overconfident):** 예측 확률보다 실제 정답 비율이 낮은 경우 (그래프가 대각선 아래로 내려감). 예를 들어, 40% 확률로 예측했지만 실제로는 30%만 정답이었다.
- **활용:** 모델이 어떤 확률 구간에서 과신하거나 과소신하는지 파악하는 데 유용하다.
- **단점:** 클래스별로 따로 그려야 하므로 다중 클래스 분류에서는 실용성이 떨어진다.

1.2.3. 보정 측정 방법: 수치적 지표

신뢰도 다이어그램을 기반으로 모델의 보정 상태를 하나의 숫자로 요약하는 지표들이다.

1. 예상 보정 오차 (Expected Calibration Error, ECE)

- **정의:** 완벽하게 보정된 모델(대각선)과 실제 모델의 신뢰도 다이어그램 간의 절대적인 거리의 가중 평균이다.
- **계산 방식:** 각 구간(bin)별 오차에 해당 구간에 속한 데이터의 수를 가중치로 부여하여 평균을 낸다.
- **목표:** ECE 값이 낮을수록 보정이 잘 된 모델이다.
- **특징:** 정확도 향상과 함께 ECE가 감소하는 경향을 보이지만, 정확도가 정체되어도 ECE는 개선될 수 있다.

2. 최대 보정 오차 (Maximum Calibration Error, MCE)

- **정의:** 완벽하게 보정된 모델과 실제 모델 간의 절대적인 거리 중 가장 큰 값이다.
- **목표:** MCE 값이 낮을수록 모델의 최악의 경우(worst-case scenario)에도 보정이 잘 유지됨을 의미한다.
- **활용:** 모든 확률 예측이 정확해야 하는 경우에 중요하다.

3. 평균 제곱근 보정 오차 (Root Mean Squared Calibration Error, RMSCE)

- **정의:** ECE와 유사하지만, L1 거리 대신 L2 거리(제곱)를 사용한다.
- **계산 방식:** 각 구간별 오차를 제곱하여 평균을 낸 후 제곱근을 취한다.

- **특징:** 작은 오차의 합보다 큰 오차 하나에 더 큰 페널티를 부여한다.
- **구간(Bin) 개수의 중요성:**
 - 구간 수가 너무 적으면 보정 오차가 작게 나올 수 있지만, 모델의 세밀한 보정 상태를 파악하기 어렵다.
 - 구간 수가 너무 많으면 각 구간의 데이터 수가 부족해져 측정값이 노이즈에 민감해질 수 있다.
 - 적절한 구간 수를 선택하는 것이 중요하다.

1.2.4. 다중 클래스 분류에서의 보정 측정

- **Argmax 기반 보정:** 예측된 클래스(가장 높은 확률을 가진 클래스)의 확률만 고려하여 보정하는 방식이다.
 - 전체 클래스 중 가장 높은 확률을 가진 클래스에 대해서만 신뢰도 다이어그램을 그린다.
 - 예측 시스템에서 최종 예측 클래스만 중요할 때 유용하다.
- **모든 클래스 보정:** nPlan과 같이 모든 클래스의 확률 분포가 중요한 경우, 각 클래스별 보정을 고려해야 한다.
 - **1-대-모든(One-vs-All) 방식:** 각 클래스를 '참(True)'으로, 나머지 클래스를 '거짓(False)'으로 간주하여 이진 분류처럼 신뢰도 다이어그램을 그린다.
 - 장점: 각 클래스의 보정 상태를 상세히 파악할 수 있다.
 - 단점: 클래스 수가 많으면(예: ImageNet) 개별 다이어그램을 분석하기 어렵다.
 - **전체 소프트맥스 출력 통합:** 모든 클래스의 소프트맥스 출력을 합쳐 하나의 신뢰도 다이어그램으로 그린다.
- **지표의 통합:**
 - ECE, MCE 등의 수치적 지표도 클래스별로 계산하거나 전체를 통합하여 계산할 수 있다.
 - 클래스별 데이터 수에 따라 가중 평균을 적용할 수 있다.
- **선택 기준:** 어떤 보정 방식을 사용할지는 모델의 목적(전체 클래스 보정 vs. 특정 클래스 보정)과 분석의 상세도 요구 수준에 따라 달라진다.

1.3. 보정 방법론

1.3.1. 이진 분류 보정 방법

- **목표:** 훈련된 모델의 예측 확률을 실제 확률에 가깝게 조정한다.
- **핵심 아이디어:** 모델의 출력(확률 또는 로짓)에 함수를 적용하여 보정한다.
- **주의사항:** 보정은 일반적으로 모델 훈련 후에 수행되며, 별도의 검증(validation) 데이터셋을 사용하여 파라미터를 학습하고 테스트(test) 데이터셋으로 성능을 평가한다.

1. 등방성 회귀 (Isotonic Regression)

- **개념:** 비모수적(non-parametric) 방법으로, 예측 확률과 실제 확률 간의 관계에 단조 증가하는(monotonic increasing) 함수를 피팅한다.
- **작동 방식:** 신뢰도 다이어그램의 구간별 데이터를 기반으로, 예측 확률을 실제 확률로 변환하는 함수 f 를 학습한다.
- **적용:** 학습된 함수 f 를 모델의 최종 출력에 적용한다.
- **구현:** scikit-learn의 `IsotonicRegression` 함수를 사용한다.
- **결과:** 신뢰도 다이어그램을 개선할 수 있지만, 모든 구간의 변동을 완벽하게 포착하지 못할 수 있다.

2. 플랫 스케일링 (Platt Scaling)

- **개념:** 로짓(logits, 최종 활성화 함수 이전의 출력값)에 시그모이드(sigmoid) 함수를 적용하여 확률을 예측하는 방식이다.
- **작동 방식:** 로짓 값에 두 개의 파라미터(a , b)를 곱하고 더한 후 시그모이드 함수를 통과시켜 보정된 확률을 얻는다.
 - $P(y=1 | x) = \sigma(az_i + b)$ $P(y=1 | x) = \sigma(az_i + b)$ $P(y=1 | x) = \sigma(az_i + b)$
(여기서 z_i 는 로짓 값, σ 는 시그모이드 함수)
- **학습:** 검증 데이터셋을 사용하여 음의 로그 우도(negative log-likelihood)를 최소화하는 최적의 파라미터 a 와 b 를 찾는다.
- **적용:** 학습된 파라미터로 로짓 값을 변환한 후 시그모이드 함수를 적용하여 보정된 확률을 얻는다.
- **결과:** 등방성 회귀보다 더 나은 보정 성능을 보일 수 있으며, ECE 값을 크게 개선할 수 있다 (예: 0.15에서 0.02로 감소).

1.3.2. 다중 클래스 분류 보정 방법

플랫 스케일링을 확장한 방법들이 주로 사용된다.

1. 온도 스케일링 (Temperature Scaling)

- **개념:** 가장 간단한 보정 방법 중 하나로, 모든 클래스에 대해 단일 온도 파라미터(t)를 학습한다.
- **작동 방식:** 소프트맥스 함수를 적용하기 전에 로짓 값을 온도 파라미터 t 로 나눈다.
 - $P_i = \frac{\exp(z_i/t)}{\sum_j \exp(z_j/t)}$ ($P_i = \frac{\exp(z_i / t)}{\sum_j \exp(z_j / t)}$) (여기서 z_i 는 로짓 값)
- **특징:**
 - 단 하나의 파라미터(t)만 학습하므로 계산이 간단하다.
 - 최대 소프트맥스 값(예측 클래스)과 정확도를 변경하지 않는다.
 - 온도 t 가 높을수록 소프트맥스 출력이 넓게 퍼지고(엔트로피 증가), 낮을수록 집중된다(엔트로피 감소).
- **적용:** 검증 데이터셋에서 음의 로그 우도를 최소화하는 t 값을 찾는다.
- **결과:** 신뢰도 다이어그램이 크게 개선되며, ECE 값도 감소한다.

2. 행렬 및 벡터 스케일링 (Matrix and Vector Scaling)

- **개념:** 플랫 스케일링을 다중 클래스 문제로 확장한 방법이다.
- **행렬 스케일링 (Matrix Scaling):** 로짓 값에 행렬 W 와 벡터 b 를 곱하여 선형 변환을 적용한다.
 - $P_i = \frac{\exp(Wz_i + b)}{\sum_j \exp(Wz_j + b)}$ ($P_i = \frac{\exp(Wz_i + b)}{\sum_j \exp(Wz_j + b)}$)
 - 클래스 수가 많아지면 행렬 W 의 크기가 제공으로 증가하여 비효율적일 수 있다.
- **벡터 스케일링 (Vector Scaling):** 행렬 W 를 대각 행렬로 제한하여 파라미터 수를 줄이고 계산을 단순화한다.
 - 행렬 스케일링의 유연성과 과적합(overfitting) 가능성 사이의 균형을 맞춘다.
- **적용:** 검증 데이터셋에서 최적의 행렬 W 와 벡터 b 를 학습한다.
- **결과:** 온도 스케일링보다 더 복잡한 보정이 가능하며, 특정 상황에서 더 나은 성능을 보일 수 있다.

1.3.3. 보정 구현 시 고려사항

- **검증 데이터셋 사용:** 보정 파라미터 학습에는 반드시 별도의 검증 데이터셋을 사용해야 한다. 테스트 데이터셋을 사용하면 과적합되어 실제 성능을 과대평가하게 된다.
- **모델 구조:** 보정은 모델 훈련 후 적용되는 후처리(post-hoc) 단계이다.
- **구간(Bin) 선택:** 신뢰도 다이어그램 및 관련 지표 계산 시 구간의 개수는 결과에 큰 영향을 미친다. 일반적으로 구간당 최소 1,000개의 샘플을 목표로 하되, 데이터셋의 특성과 분석 목적에 따라 조정해야 한다.
- **데이터 불균형:** 클래스별 데이터 수가 크게 다를 경우, 가중 평균을 사용하거나 각 클래스를 개별적으로 분석하는 것이 좋다.
- **모델 복잡성:** 일반적으로 모델이 복잡할수록(깊이, 너비 증가 등) 보정 성능이 떨어지는 경향이 있다.
- **RNN과 DNN:** RNN과 같은 순환 신경망도 DNN과 마찬가지로 보정이 필요하며, 온도 스케일링과 같은 방법이 효과적일 수 있다.

1.4. 보정의 실제 적용 사례 및 논의

1.4.1. nPlan의 활용 사례

- **건설 프로젝트 예측:** nPlan은 건설 프로젝트의 각 활동별 완료 시점을 확률적으로 예측한다.
- **몬테카를로 시뮬레이션:** 예측된 확률 분포를 바탕으로 몬테카를로 시뮬레이션을 수행하여 프로젝트 전체의 완료 시점 분포를 파악한다.
 - 이를 통해 프로젝트 지연 가능성이나 예상 완료 시점을 50% 또는 90% 확률로 제시할 수 있다.
- **데이터 소스:** 주로 건설 회사와의 계약을 통해 데이터를 확보하며, 데이터는 보안이 유지된다.
- **비즈니스 모델:** B2B 방식으로 건설 회사에 예측 및 분석 서비스를 제공한다.

1.4.2. 보정 관련 질의응답 및 심층 논의

- **보정이 필요한 경우:** 모델의 출력이 실제 확률 분포를 정확하게 반영해야 할 때, 또는 의사결정 과정에서 모델의 신뢰도를 중요하게 고려해야 할 때 필요하다.
 - 예: 의료 진단, 금융 위험 평가, 건설 프로젝트 예측 등.

- **보정의 성능 영향:** 보정 방법은 모델의 정확도(F1 점수, AUC 등)에 영향을 줄 수 있으며, 때로는 개선되기도 하지만 때로는 악화될 수도 있다. 이는 모델의 예측 클래스 자체를 변경하는지에 따라 달라진다.
- **다차원적 보정:**
 - 일반적인 보정은 전체 예측 확률에 대해 수행되지만, 특정 입력 특징(feature)에 따라 모델의 보정 상태가 달라질 수 있다는 질문이 제기되었다.
 - 예를 들어, 전체적으로는 보정되었더라도 특정 입력값($x=3$)에서는 60%, 다른 입력값($x=5$)에서는 80%의 확률을 보일 수 있는데, 이를 더 세밀하게 분석하고 보정할 수 있는지에 대한 논의가 있었다.
 - 이는 입력 공간을 여러 차원으로 나누어 각 부분 공간별로 보정을 수행하는 방식(예: 의사결정 트리 활용)으로 확장될 수 있으나, 아직 문헌에서 흔히 다루어지지 않는다고 언급되었다.
- **이상치(Anomaly) 처리:** 모델의 예측이 일반적인 데이터 분포에서 벗어난 이상치에 대한 것일 경우, 보정보다는 해당 데이터를 별도로 처리하거나 추가적인 불확실성 지표를 활용하는 것이 더 적절할 수 있다.
- **보정과 훈련의 관계:** 보정은 일반적으로 훈련 후 적용되는 후처리 단계이지만, 일부 연구에서는 특정 훈련 방법이 더 잘 보정된 모델을 생성할 수 있다고 제안하기도 한다.
- **기존 모델의 보정 파라미터:** 모델 자체에 이미 온도와 같은 보정 관련 파라미터가 포함되어 있더라도, 추가적인 보정 단계를 적용할 수 있다.
- **머신러닝 커뮤니티:** KDD 2020의 발표자들은 자신들의 지식과 경험을 공유하고, 다른 참가자들과의 토론을 통해 배우는 것을 중요하게 생각한다.
 - 주간 머신러닝 페이퍼 클럽 운영, 온라인 발표 및 토론 등을 통해 지식을 공유한다.